MaskPrint: Take the Initiative in Fingerprint Protection to Mitigate the Harm of Data Breach

Yihui Yan*†
School of Information Science and Technology
ShanghaiTech University
Shanghai, China
yanyh@shanghaitech.edu.cn

Zhice Yang
School of Information Science and Technology
ShanghaiTech University
Shanghai, China
yangzhc@shanghaitech.edu.cn

ABSTRACT

The privacy of fingerprints is a growing concern due to the risk of data breaches and subsequent attacks. A key issue is that the information of the same fingerprint may exist on multiple devices, and device-level protection mechanisms are fundamentally limited in their coverage. Consequently, information leakage from any device potentially affects all enrolled fingerprint recognition devices. In this paper, we introduce a novel fingerprint enrollment method called MaskPrint, which allows users to enroll in various fingerprint recognition systems using distinct information. This approach can largely mitigate the risk of data breaches, providing a user-transparent, device-agnostic fingerprint protection measure. Our method involves collecting the original fingerprint information, selecting a minimum feature set, synthesizing protective fingerprints, and fabricating physical ones for enrollment. Users can complete these procedures on their own. We validate the effectiveness and usability of MaskPrint on commercial fingerprint recognition systems.

CCS CONCEPTS

• **Security and privacy** → **Biometrics**; **Privacy protections**; *Spoofing attacks*; *Usability in security and privacy*.

KEYWORDS

Fingerprint, Biometrics, Privacy Protection

ACM Reference Format:
Yihui Yan and Zhice Yang. 2024. MaskPrint: Take the Initiative in Fingerprint Protection to Mitigate the Harm of Data Breach. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS '24), October 14–18, 2024, Salt Lake City, UT, USA.* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3658644.3670364

*Also with Shanghai Institute of Microsystem and Information Technology, Chinese Academy of Science.
†Also with University of Chinese Academy of Sciences.
Corresponding Author: Zhice Yang.

CCS '24, October 14–18, 2024, Salt Lake City, UT, USA

ACM ISBN 979-8-4007-0636-3/24/10.
https://doi.org/10.1145/3658644.3670364

1 INTRODUCTION

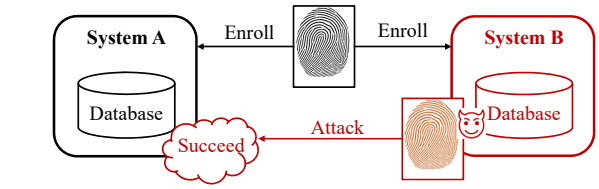
There is a long history of identifying individuals using fingerprints. Fingerprint has been used to sign contracts before the Common Era because it is hard to forge and nearly unique and durable over an individual's lifetime. Nowadays, automatic fingerprint recognition systems (FRSs) are widely deployed for access control, mobile payment, attendance tracking, *etc.*, which facilitate many aspects of our lives.

Along with this, the privacy of fingerprints has become a major concern for users. FRSs need to collect and store users' fingerprint information for comparison and identification. Once this information is leaked, it can be exploited for injection attacks [17] or forging fake fingerprints to launch presentation attacks [24, 32], leading to authentication bypass [6] and even significant financial losses [7, 11], as reported in various cases. As such, fingerprint data has always been a high-value target for attackers. A few widely publicized cases include the theft of 5.6 million people's fingerprints from the government's database in 2015 [10], and the leakage of 1 million fingerprint data from a commercial physical access control system's database in 2019 [1].

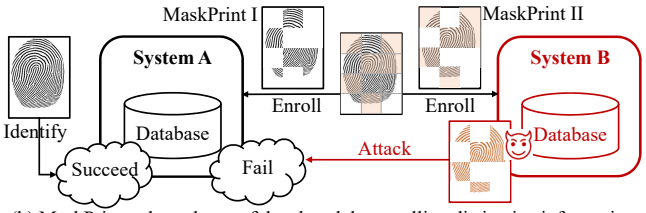
To contain these risks, both industry and academia have long devoted efforts to secure fingerprint data. For example, since version 6.0, Android natively supports fingerprint recognition. It utilizes the ARM TrustZone mechanism to isolate all I/O and processing modules associated with fingerprint data. Cancelable fingerprint templates [31] and fingerprint cryptosystems [18] are proposed to store fingerprints in a non-invertible form so that even if the data is leaked, adversaries cannot recover enough information for attacks.

Despite these efforts having secured hardware, algorithms, storage, and various aspects along the fingerprint data flow, there are still gaps in realizing a full-path trustworthy protection solution. One aspect is that users lack proper means to audit whether protection mechanisms have been responsibly implemented by vendors. Another, as shown in Figure 1(a), more fundamental issue is that users tend to enroll the same fingerprint in different FRSs. Then, if one system's data is compromised, it threatens all associated systems, regardless of the protection mechanisms they employ. In other words, the security of fingerprint data is determined by the FRS using the weakest protection mechanism. However, enforcing consistent and strong protection mechanisms on every FRS, including legacy devices, is barely impractical.

The root cause of this challenging situation lies in the fact that the source of fingerprint information is not the device but the user's body. Every enrolled FRS device retains an identifier derived from



(a) Data breach of one fingerprint recognition system threatens other systems.



(b) MaskPrint reduces harm of data beach by enrolling distinctive information.

Figure 1: MaskPrint Secures Fingerprint at the Pre-enrollment Stage. (a) If a fingerprint is enrolled in different fingerprint recognition systems, a breach of any system's database can pose security risks to other systems. (b) MaskPrint allows users to register different fingerprint "fragments" in different recognition systems. These fragments are mutually distinctive but similar to the original fingerprint, thus mitigating the harm of fingerprint database breaches without impacting usability.

the *same* fingerprint, and this information correlation makes on-device data protection practices always face security risks beyond their own protection scope. To this end, in this paper, we propose converging protection measures to the fingerprint source at the pre-enrollment stage, as a necessary supplement to current fingerprint information protection mechanisms.

Specifically, we propose a fingerprint enrollment scheme called MaskPrint. It allows users to use distinctive information to enroll in different FRSs. A simplified example is shown in Figure 1(b). The benefit is that even if some systems experience data breaches, it will not directly compromise the security of other systems, thus rendering a user-aware, device-agnostic fingerprint protection measure. To put it into practice, we have to address the following issues:

Firstly, the enrolling method should not confront normal fingerprint recognition usage. The user should be able to use their own finger to pass through the enrolled FRSs as usual. The feasibility of this lies in an important observation: practical FRSs are designed with considerable redundancy, and their matching algorithms only require a small subset of fingerprint features to make accurate decisions (Section 4). Based on this finding, we first collect and extract the full feature set from the original fingerprint, and then use dedicated feature selection and masking algorithms to synthesize "masked" fingerprints, *i.e.*, MaskPrints, for enrollment (Section 5.1). These MaskPrint images contain subsets of the original fingerprint features, thus allowing them to match the original fingerprint during identification; yet, they have minimal intersection with each other, thus preventing leakage of others' information.

The follow-up issue is how to utilize the generated MaskPrint information. In practice, FRSs take images of the physical fingerprints as input; few can be enrolled with provided fingerprint images. Similar to the fake fingerprints used in presentation attacks [8, 15, 24], our approach is to actualize the MaskPrint information. However, considering the difficulty, cost, and accuracy, we invent a new method for fabricating physical fingerprints using a laser printer and liquid silicone (Section 5.2). Using this method, physical MaskPrints can be massively produced with high precision.

To use MaskPrint, users only need to take a one-time action to collect fingerprint information, generate MaskPrint images using the MaskPrint software, and then fabricate physical MaskPrints. These procedures can be fully completed by themselves or outsourced to a trusted third party. When encountering new fingerprint devices, they can use their physical MaskPrints as fingerprints to enroll and protect their fingerprint information (Section 5.3). Apart from the enrollment, the fingerprint devices are used as usual.

This paper contributes to the area of fingerprint privacy protection in the following aspects:

- We propose a method to protect fingerprint information at the pre-enrollment stage, which is transparent to users and device-agnostic.
- We identify the redundancy in fingerprint matching algorithms and propose a smart masking scheme for minimizing the enrolling information.
- We present a high-precision and cost-effective method for fabricating physical fingerprints.
- We thoroughly evaluate MaskPrint with commercial-off-the-shelf (COTS) products. The results indicate it can achieve a good balance between privacy protection and usability.

2 BACKGROUND

2.1 Fingerprint Recognition System

As shown in Figure 2, a typical fingerprint recognition system has two working modes, enrollment and identification. Both modes first capture a fingerprint image from the sensor and generate a fingerprint template. The template contains distinctive features of a fingerprint, which are used to uniquely determine the user's identity. After converting a fingerprint image to a fingerprint template, the enrollment mode stores the template in the fingerprint database, while the identification mode compares the template of a newly captured image against the enrolled templates, and outputs whether it is matched with one of the enrolled templates.

2.2 Fingerprint Template

A fingerprint template is a mathematical representation of distinctive features in the fingerprint images, which is more efficient at storage and processing. The features described in a template can be the fingerprint minutiae, like bifurcation and ridge ending (as shown in Figure 2), or some special descriptors, like those extracted by the SIFT (Scale Invariant Feature Transformation) algorithm [39] and DNN (Deep Neural Network) [19]. According to the statistics presented on FVC-onGoing [4] (an online evaluation system for fingerprint recognition algorithms), most of the top-ranked algorithms are minutiae-based, thus, we only consider minutiae-based fingerprint recognition algorithms in this paper.

For a minutiae-based algorithm, the features are typically the coordinate (x, y), direction (θ) and type (bifurcation or ridge ending) of the minutiae (m). An example of these features is shown in Figure

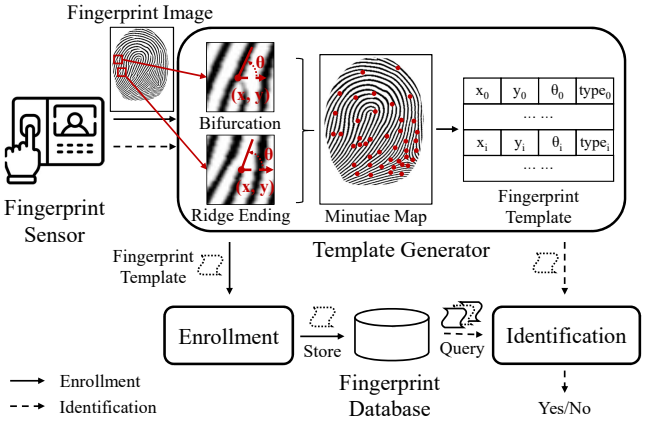


Figure 2: Architecture of a Typical Fingerprint Recognition System (FRS).

2.[1] Then, a template with n minutiae can be expressed as,

$$T = \{m_i = (x_i, y_i, \theta_i, type_i) | i = 1, 2, ..., n\},$$

where the m_i is the i^{th} minutia. In practice, a template may contain other information, such as user ID and statics of the recorded minutiae, in this paper, without loss of generality, we treat the template as a set of minutiae.

2.3 Template Matching

As finger placement can bias the absolute value of minutia coordinate and direction, the relatively stable geometric relationships, especially spatial distance and direction difference, among the minutiae are used to match the templates. Take Bozorth algorithm[9] as an example, given a template T, its relative geometric structures (G) of each minutiae pair can be expressed as (also see Figure 3):

$$G = \{\langle m_i, m_j \rangle = (d_{ij}, \beta_i, \beta_j, i, j) \mid i \neq j\},$$

where, $\langle m_i, m_j \rangle$ is the pair pointing from the i^{th} minutiae to the j^{th} minutiae, d_{ij} is the spatial distance between them, β_i and β_j are the angles relative to the line connecting the minutiae pair.

The comparison of two templates is to compare their minutiae pairs. Since the skin would be stretched or compressed with various pressing force, the relative distance and direction also vary in different images. Thus, tolerance boxes are considered. For example, if two minutiae pairs ($\langle m_i^A, m_j^A \rangle$ and $\langle m_p^B, m_q^B \rangle$) from two templates T^A and T^B satisfy the conditions:

$$|d_{ij}^A - d_{pq}^B| \leq d_0, \text{ and}$$
$$\min(|\beta_i^A - \beta_p^B|,\ 2\pi - |\beta_i^A - \beta_p^B|) \leq \theta_0, \text{ and}$$
$$\min(|\beta_j^A - \beta_q^B|,\ 2\pi - |\beta_j^A - \beta_q^B|) \leq \theta_0,$$

where d_0 and θ_0 are the distance and direction tolerance boxes, then, $\langle m_i^A, m_j^A \rangle$ and $\langle m_p^B, m_q^B \rangle$ are considered *compatible*, *i.e.*, they are matched. After comparing all minutiae pairs from two templates T^A and T^B, a *matching score* is calculated according to the matched minutiae pairs. When the score is higher than a given threshold, two templates are considered to come from the same fingerprint.

[1] Although the coordinate and direction might have a different definition in various systems, (*e.g.*, clockwise or counterclockwise), the core idea of how to match templates remains the same and is presented in Section 2.3.

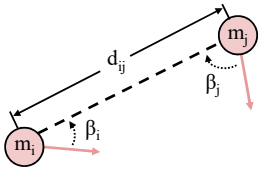


Figure 3: Definition of Relative Geometric Structure of a Minutiae Pair. The circles are at absolute positions x, y of the minutiae. The arrows on the circle represent their directions θ. d_{ij} is the distance between minutiae m_i and m_j, and β_i and β_j are the angles of m_i and m_j relative to the line connecting them.

The geometric relationships described above are the basis of fingerprint template matching. For specific FRS algorithms, other attributes, such as the ridge frequency and curvature around a minutia [21, 23], may be included to improve the performance.

2.4 Fingerprint Reconstruction

Plenty of work has proposed that the fingerprint image can be reconstructed from a fingerprint template [13, 14, 22, 37]. The reconstructed image contains identical minutiae described by the template, and can match the original fingerprint image. Hence, the reconstructed image can pass the matching of not only the system where the template is leaked from, but also other systems enrolled with the same original fingerprint.

3 THREAT MODEL

We assume that a user needs to enroll in multiple FRSs, *e.g.*, the attendance machine, the door access controller, and the bank fingerprint authentication system. We further assume that the user tends to enroll in these FRSs using the same finger, *e.g.* the right index finger, as it may be the dominating one.

We assume that the adversary wants to bypass one or several FRSs that the user has enrolled in. We assume the adversary is able to obtain the user's enrolled fingerprint information from one of the FRS databases, *e.g.*, through network attacks. The adversary can exploit the leaked information to launch injection [17] and presentation attacks [24, 32].

Our security goal is to reduce information leakage from compromised FRS databases, preventing adversaries from immediately obtaining valid user fingerprint information to compromise other enrolled FRSs. Our security goal does not encompass other means of fingerprint leakage beyond FRS databases, such as offline collection [6] or finger photos on social media [5]. These means are either limited in scalability or avoidable by users. Additionally, our security goal does not include healing leaked fingerprint information[2] or limiting leakage from FRSs that have not adopted our protection measures.

[2] By far, there are not many remediation means since fingerprints almost never change and the adversary can present information to FRSs that closely resembles genuine fingerprints. Spoofing detection [32] helps in verifying whether fingerprint images originate from a live person, but many FRSs lack this capability, which falls outside the scope of this paper.

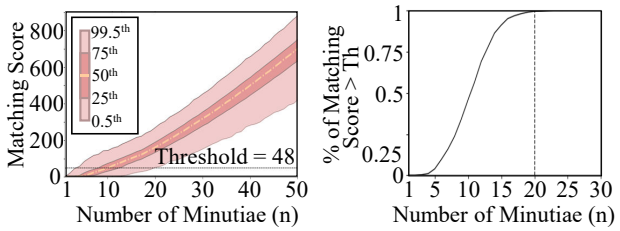


Figure 4: Fingerprint Recognition Systems Contain Redundancy. (a) When the input is from the same finger, the matching score increases with the increase of the number of minutiae contained in the enrolled template. (b) In most cases, the enrolled template only needs to contain 20 minutiae for recognition.

4 INSIGHT OF MASKPRINT

As a commercial product facing a broad user base, FRSs have to consider various input situations. As dry/wet/wounded fingers might lead to non-ideal impressions (see Figure 16 in Appendix A), FRS matching algorithms can tolerate distorted, missing, and spurious minutiae caused by the input noise. As a result, the input fingerprint can be matched even with a small overlap with the template. This raises an interesting question: Is it possible to use only part of the fingerprint information as the template and still achieve fingerprint recognition?

To answer this question, we measure the matching results of a fingerprint by varying the number of minutiae contained in its template in the FRS. The experiments are conducted with VeriFinger, a commercial FRS product [12].

Specifically, 30 images of the same fingerprint are collected. Each image contains about 55 minutiae depending on the input condition of the finger. We choose one of the images and convert it to a template using the VeriFinger software. Then, we construct a new but shrunken "template" of the fingerprint by randomly selecting n minutiae from the template. We store the new template in the emptied FRS database and use other input images to test identification, *i.e.*, calculate the matching score, against it. A higher score indicates a higher similarity between two fingerprints. We repeat the above steps to construct new enrolled templates, and record their matching scores.

The vertical axis and color depth of Figure 4(a) display the distribution of matching scores at a given n. The horizontal axis of Figure 4(a) illustrates the trend of increasing matching scores as the number of minutiae n included in the new template increases. When the template contains 50 minutiae, most matching scores are larger than 400, which is far beyond the default matching threshold of the FRS, *i.e.*, 48.

Additionally, Figure 4(b) re-plots the results to emphasize the proportion of input images with a matching score exceeding the threshold. It shows that, for this FRS, only 20-30 minutiae in the enrolled template are sufficient to correctly match the input fingerprint.

4.1 Idea of Masking Fingerprint

Based on the above observation, the key idea of MaskPrint is leveraging the redundancy in FRS matching designs. Since the enrolled template only needs a small subset of the full minutiae set to evaluate test inputs, it is possible to reduce the number of minutiae contained in the templates stored in FRSs while maintaining the recognition functionality. Such margin enables us to flexibly choose the features included in a certain template. By carefully selecting or masking out specific minutiae, we can derive protective templates, MaskPrints, each matching the full original fingerprint but with sufficient mutual distinctions.

As such, we propose to enroll in different FRSs using different MaskPrints. The benefit is, that even if one FRS experiences a data breach, the leaked information cannot be exploited to compromise other FRSs, as they are using distinctive templates. Meanwhile, users can use their fingers to pass the identification, as usual.

5 DESIGN

An overview of MaskPrint is shown in Figure 5. As a protective fingerprint enrollment method, its usage process consists of three steps. As outlined in Figure 5(a), the user first uses a fingerprint scanner to collect the fingerprint images. The MaskPrint software turns these images into MaskPrint images (Section 5.1). Secondly, the user fabricates physical MaskPrints according to the MaskPrint images (Section 5.2). After that, the user takes the physical MaskPrints to enroll in different FRSs (Section 5.3).

5.1 MaskPrint Generation

This section introduces how MaskPrint images are generated, which involves three parts, image acquisition, SuperDB establishment, and mask generation. They are outlined in Figure 5(b).

5.1.1 Image Acquisition. MaskPrint is to protect fingerprint information, hence the first step is to collect the user's fingerprint images. We propose to use optical fingerprint scanners, as they are cost-effective and easy-to-use tools for capturing fingerprint images.

An optical fingerprint scanner is essentially an optical imaging system, similar to cameras. Like typical cameras, the output images are uncalibrated and may contain imaging distortions, such as those caused by the lens. These distortions can change the coordinate and direction of minutiae. If these distortions are left untreated, they will persist and be contained in the following information we will work on, thus affecting the final matching performance.

Typically, imaging systems can be calibrated using a chessboard [38], but the fingerprint scanner cannot capture the chessboard printed on paper. The reason is rooted in the imaging mechanism of the scanner, which we will explain in detail shortly in Section 5.2. To solve this problem, we utilize the method in Section 5.2 to fabricate a chessboard with silicone that can be captured by the scanner. The imaging distortions can be calibrated by referring to the distortions of the scanned chessboard.

5.1.2 SuperDB Establishment. After obtaining fingerprint images using a scanner, we found it impossible to capture all the minutiae of a finger in a single input. To obtain more comprehensive fingerprint information, we collect multiple captures of the same finger from different positions. However, this approach introduces a problem: due to variations in positioning and pressure during different captures, the minutiae information of different captures

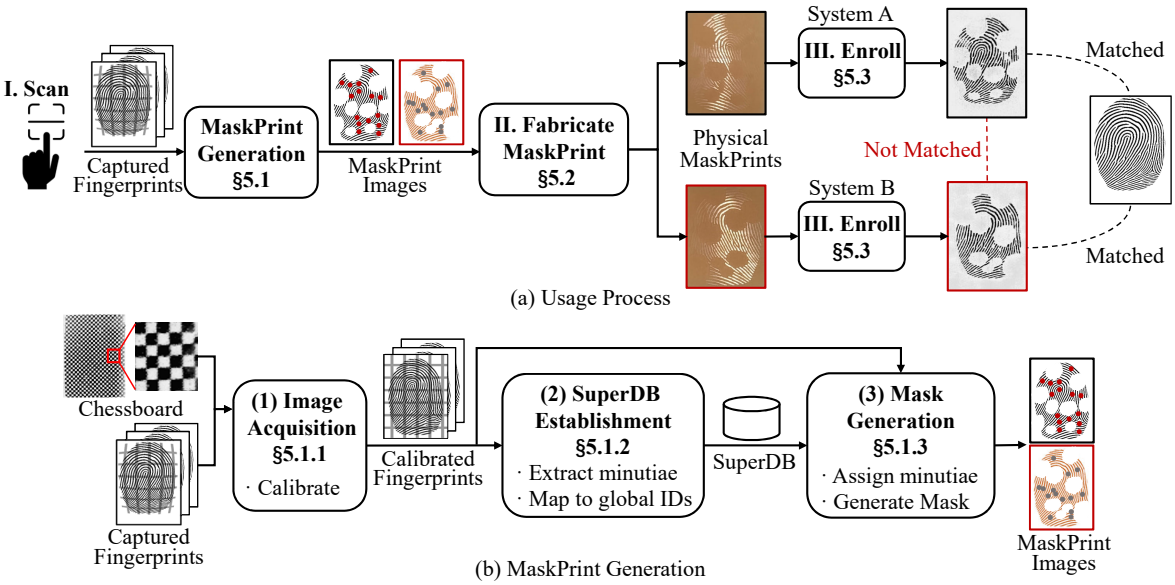


Figure 5: MaskPrint Overview. (a) MaskPrint involves three user operations, I. scan fingerprints with fingerprint sensor, II. fabricate physical MaskPrints, and III. enroll fingerprint recognition systems with physical MaskPrints. Different MaskPrints match the original finger, but they cannot match each other. Subfigure (b) outlines the procedures to generate MaskPrint images from the sensor input.

differs. To address this issue, we introduce SuperDB, which synthesizes information from multiple fingerprint images to provide a complete view of the fingerprint.

SuperDB is based on the concept of super-templates introduced in [36]. SuperDB stores the coordinates of all minutiae in the images and assigns the minutiae from different images that should originate from the same minutia on the finger with the same global ID. Each record of the SuperDB can be expressed as,

$$\{ID_{image},\ ID_{global},\ x,\ y\}.$$

For a template converted from image ID_{image}, its minutia at coordinate (x, y) is assigned to the global minutiae index ID_{global}. Minutiae across different images with the same ID_{global} are considered the same. Populating the SuperDB is to calculate the similarity of minutiae to assign their ID_{global}.

One approach is to refer to the compatibility assessment method of two minutiae pairs in Section 2.3 to determine the compatibility of two minutiae. However, if only considering two pairs, there may be significant errors. Therefore, our approach is to statistically analyze all pairs associated with the two minutiae to make decisions.

The algorithm can be explained through the example in Figure 6. Figure 6(a)-(b) are two templates (T^A and T^B), both containing four minutiae, $\{m^A_{0-3}\}$ and $\{m^B_{0-3}\}$. According to the matching conditions in Section 2.3, after traversing all minutiae pairs between two templates, four groups of compatible pairs are found and shown in Figure 6(c). Although m^A_0 and m^B_2 are not the same minutia, $\langle m^A_0, m^A_2\rangle$ and $\langle m^B_2, m^B_3\rangle$ are matched due to coincidentally similar structure.

To distinguish such mismatches, our intuition is that if one minutiae is compatible with another, it is likely that all minutiae pairs associated with them are also compatible. Therefore, we record the occurrence for each minutiae in all compatible minutiae pairs in a compatibility matrix C. As shown in Figure 6(d), as $\langle m^A_1, m^A_2\rangle$ and $\langle m^B_0, m^B_1\rangle$ are compatible, $C[m^A_1, m^B_0]$ and $C[m^A_2, m^B_1]$ are increased by 1. In this sense, the value of the compatibility matrix represents the confidence of assessing two compatible minutiae. In this simple example, $m^A_1 \sim m^B_0$, $m^A_2 \sim m^B_1$, and $m^A_3 \sim m^B_3$ are considered as compatible minutiae since their matrix elements exhibit prominent values. They will be assigned with same ID_{global}s.

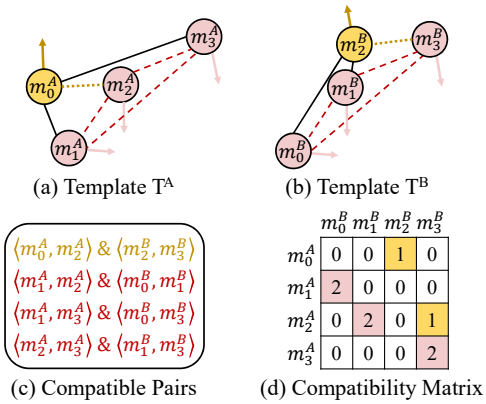


Figure 6: Matching Minutiae of Two Templates. To verify if minutiae from two different templates (a) and (b) are derived from the same minutia, (c) our approach is to verify the compatibility of all minutiae pairs associated with the two minutiae. (d) This information is counted and stored in the compatibility matrix of the two templates.

To add the minutiae of a new template to the SuperDB, we first find the compatible minutiae already in the SuperDB and assign them with the corresponding ID_{global}s. Other unmatched minutiae are assigned with new ID_{global}s. Before completing the population of SuperDB, we also clean out minutiae whose ID_{global}s only appear once or twice. We consider them as spurious minutiae caused by imaging noise.

Figure 7: Impact of Mask Size. The minutia marked with a red dot in (a) is masked with circle masks. By increasing the size of the mask (from (b) to (d)), the masked minutia becomes harder to recover.

Figure 8: Impact of Close Neighbours. (a) When the red-dot-marked minutia needs to be masked out, while two yellow-dot-marked minutiae are selected to keep, (b) the mask can not retain its designed size. (c) By masking close neighbors together, all three minutiae are well masked.

5.1.3 Mask Generation. To ensure that different FRSs have distinct enrolled information, our approach is to utilize different subsets of the complete minutiae set of the finger, *i.e.*, SuperDB, for enrollment. Employing isolated minutiae subsets guarantees information distinctiveness between FRSs. Note that while adding spurious minutiae to each subset can increase distinctiveness, it also introduces differences with genuine fingerprints and harms usability. Therefore, we opt to directly create a mask on a fingerprint image to wipe out all non-selected minutiae to generate the MaskPrint image.

The following aspects need to be considered in order to generate appropriate masks:

(a). Mask Size. Hiding minutiae information requires a mask of sufficient size. In most areas of a fingerprint, the ridge direction and frequency change slowly and almost continuously, hence a masked area could be inferred based on the local texture. For example, as shown in Figure 7(b), the minutia marked in Figure 7(a) is masked by a small circle. Due to the unequal number of ridges around the circle, the masked minutia can still be inferred by extending the discontinuous ridges. After increasing the size of the mask (Figure 7(c)-(d)), it becomes challenging to recover the precise information of the minutia.

(b). Close Neighbors. As shown in Figure 8(a), when a minutia has very close neighbors, it is not possible to only mask it out. Thus, MaskPrint masks the close neighbors together to avoid fragmenting the mask (Figure 8(b)). The strategy brings additional benefits in information hiding. As shown in Figure 8(c), after masking the three minutiae, it is difficult to tell how many minutiae are beneath.

(c). Mask Shape. Since each MaskPrint only whitelists a small portion of the full minutiae set, there are overlapped masked-out areas between different MaskPrints, as shown in Figure 9(a). When using constant base shapes, such as circles, to mask minutiae, the borders of some overlapping areas might completely coincide (Figure 9(b)-(c)). Although these MaskPrint areas do not contain minutiae (as outlined by the dashed box in Figure 9(b)-(c)), advanced matching algorithms might still find similarities based on these perfectly overlapped non-minutiae areas. Therefore, as shown in

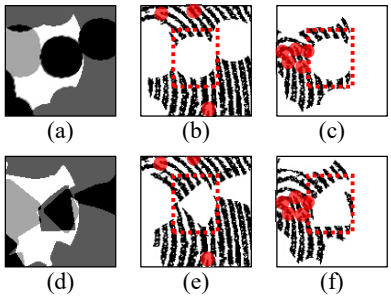


Figure 9: Impact of Mask Shape. (a) highlights the masked areas of two MaskPrints denoted with dark and light grey. Their overlap is shown in black. (b)(c) are the two MaskPrints with their minutiae shown in red circles. The constant mask shape in (b)(c) leads to identical borders in the dashed box areas, which might reduce the distinctiveness between (b) and (c). (d)(e)(f) show the case that randomizes the mask shape to focus the similarity assessment on minutiae.

Figure 9(d)-(f), our approach is to use random shapes to mask minutia, thus randomizing MaskPrint boarders and forcing matching algorithms to determine MaskPrints similarities primarily based on the minutiae they contain.

(d). Spacial Distribution. Since the minutiae around the center of the fingertip tend to have a higher chance of being captured in identification tests, a MaskPrint containing too many marginal minutiae would affect the performance of the enrolled FRS. Conversely, if too many center minutiae are included, other MaskPrints would need to incorporate more marginal minutiae. Therefore, the minutiae should be uniformly selected in space.

According to the above analysis, MaskPrint images are generated with the following steps.

1). Select Minutiae to Keep According to (b) and (d). Firstly, we classify the minutiae into two classes: central and marginal minutiae, according to the number of times an ID_{global} appears in the SuperDB. The intuition is that, if a minutia is closer to the center of the fingertip, it will appear more times than marginal ones among the collected fingerprint images. Then, we select minutiae by randomly and proportionally choosing them from the two classes. Thirdly, a selected minutia's close neighbors are also selected, thus they will not be masked together. Finally, the selected minutiae are identified by a set of ID_{global}s.

2). Generate MaskPrint Images According to (a) and (c). With the above minutiae set, we search within SuperDB, based on the ID_{image} attribute, to find the fingerprint image that contains the highest number of minutiae overlapping with the selected set. Then, we mask out all other minutiae on this image except for the selected set. The shape of the mask is randomly generated with sufficient size for each minutia to be masked. Subsequently, any remaining content outside the range of the selected minutiae is also deleted.

5.2 MaskPrint Fabrication

Few FRSs can enroll using digital images. To enroll using the generated MaskPrint images, we propose a method for fabricating physical clones of fingerprint images that can be captured by fingerprint scanners. We aim to make the fabrication precise and cost-effective.

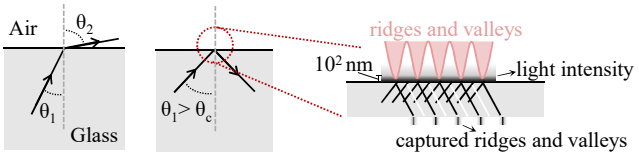


Figure 10: Imaging Mechanism of Optical Fingerprint Scanner. The scanner is based on Frustrated Total Internal Reflection (FTIR) phenomenon. It can only image objects very close to the imaging surface.

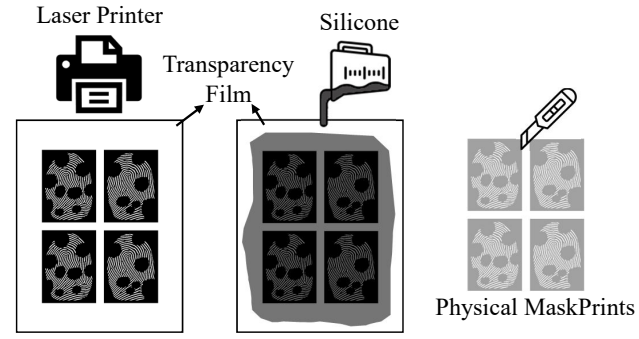


Figure 11: Physical MaskPrint Fabrication. A user can fabricate physical MaskPrints through three operations: (a) printing MaskPrints images on a transparency film with a laser printer; (b) pouring silicone on the printed content; and (c) trimming the cured silicone.

5.2.1 FTIR-Based Fingerprint Scanner. As mentioned earlier in Section 5.1.1, although optical fingerprint scanners are similar to cameras, they cannot directly image the content printed on paper due to their unique imaging mechanism—Frustrated Total Internal Reflection (FTIR).

As shown in Figure 10(a), there is a light source underneath the glass surface of the optical scanner. When light passes from glass into the air, refraction occurs. If θ_1 is larger than a critical angle (θ_c), total internal reflection occurs (Figure 10(b)). During total reflection, a layer of light close to the glass surface known as the evanescent wave is formed. When an object is close to the surface, the evanescent wave is transmitted off the glass onto and reflected by the object. This phenomenon is called FTIR. However, the amplitude of the evanescent wave falls off exponentially. Thus, only objects very close to the glass surface can reflect the evanescent wave. As the ridges of fingerprints are closer to the surface than the valleys, the intensity of light reflected by the valleys is much lower than that reflected by the ridges. Therefore, capturing the reflected light intensity can precisely image the fingerprint (Figure 10(c)).

According to FTIR, the scanner can only capture objects within a few hundred nanometers of the sensing surface [20]. This prevents us from printing MaskPrint images for enrollment because the gap between paper and glass (about 100 um [25]) far exceeds the effective distance of FTIR.

5.2.2 Fabricating Physical MaskPrints. To enable the scanner to capture fingerprints, we utilize materials with better "affinity" than paper, such as silicone, to reveal fingerprint information on the glass surface. Various fabrication methods have been reported, primarily for the purpose of spoofing FRSs. Most of these methods involve two steps: (1) creating a mold of the fingerprint, and (2) filling the mold with casting material (*e.g.*, liquid silicone) to imprint the fingerprint. The mold is often made from a live finger. If only fingerprint images are available, existing methods involve etching the fingerprint onto a Printed Circuit Board (PCB) to create a mold. However, this method is too complex and expensive for ordinary users.

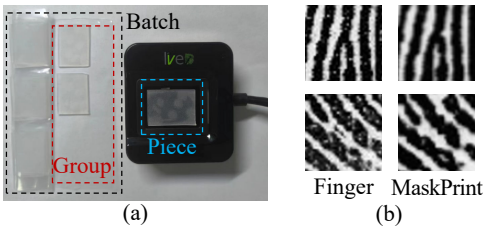


Figure 12: Example of Physical MaskPrints. (a) Using trimmed silicone MaskPrints to enroll. (b) Comparison of the scanned images of the original finger and the corresponding physical MaskPrints clone.

To this end, we propose a cost-effective yet precise fabrication method. As illustrated in Figure 11, we utilize a laser printer and smooth plastic films, such as transparencies, to create molds. The laser printer operates by melting carbon toner particles onto the film. The toner layer is a few micrometers thick, so when silicone is applied over the printed film, the cured silicone retains the thickness of the toner layer. Specifically, these minute variations can be captured by an FTIR scanner. Therefore, printing the valleys and ridges of a MaskPrint image in black and white can precisely generate a physical MaskPrint that is imagable by the scanner. Figure 11 shows the artifacts and results.

5.3 MaskPrint Enrollment

Users can enroll the selected minutiae set into FRSs using physical MaskPrints. However, a practical issue is: the minimum number of minutiae required for successful matching with real fingers may vary among different FRSs. If we fix the number of minutiae contained in each MaskPrint, it may unnecessarily contain excessive information for certain FRSs, contradicting the principle of information minimization. One approach is to measure and provide a common reference database for every FRS. Instead, we propose a more flexible and universal method - users measure the minimum required number based on the feedback from specific FRSs during enrollment.

That is, for a target FRS, the MaskPrint software generates a series of MaskPrint images. The selected minutiae sets of these images are gradually enlarged (*e.g.*, incrementally unionizing 5 more minutiae), and we refer to such a series of MaskPrints as a *batch*. Before enrollment, the user prepares all physical MaskPrints in a batch. Then, during enrollment, the user selects the MaskPrint for enrollment from the batch in the order of increasing the minutiae count. After each enrollment, the user attempts to test the real finger five times. If there is one failed identification, the MaskPrint with more minutiae is used for enrollment. The user repeats this process until the real finger can be consistently identified during tests.

Actual FRSs require the user to input multiple times during enrollment to ensure the completeness of fingerprint information. Similarly, MaskPrint software can generate multiple MaskPrint images of different ID_{image}s that meet the criteria of containing certain minutiae set (refer to Section 5.1.3) to allow the user to input the same template multiple times. Such MaskPrints containing the same minutiae set but derived from different fingerprint images are referred to as a *group*. Figure 12(a) shows a group of three such MaskPrints. During each enrollment, the user presses all three MaskPrints in the group on the scanner once.

6 EVALUATION SETUP

6.1 Implementation

We present the implementation of MaskPrint, and the cost of using it.

Image Acquisition Tool. We collect the fingerprint images with ZKTeco Live 20R, an optical fingerprint scanner whose resolution is 500 dpi. The data collection software is written in C# with the SDK provided by ZKTeco. The output images are 300 $px \times 400$ px in width and height, which corresponds to a $15.24mm \times 20.32mm$ effective sensing area.

MaskPrint Generation. Our host system is a laptop with an AMD Ryzen 7 4700U processor and runs Windows 10 operating system. Besides the image acquisition software, other parts of MaskPrint software are implemented with Python. The minutiae in the images are extracted by VeriFinger software [12]. When MaskPrint establishes the SuperDB, the tolerance boxes d_0 and θ_0 in Section 2 are set as 5 pixels and 10 degrees. If a minutiae appears in more than half of the collected images, we mark it as a center minutiae, otherwise a marginal minutiae. The randomized mask shape is generated by connecting 8 points around a center point, whose radius is uniformly distributed in [15, 25] pixels, which could cover 3-6 ridges.

MaskPrint Fabrication. An HP M427dw laser printer is used to print the toner on the transparencies with a resolution of 1200 dpi. The fingerprint images are binarized before printing to avoid a dotted texture, which is used to simulate gray-scale images. The physical MaskPrint is fabricated with silicone, which has a hardness of 20. A heated bed is used to quicken the curing reactions, which takes about 15 minutes at $50^{\circ}C$.

Cost of Using MaskPrint. It requires a laser printer ($100) and a fingerprint scanner ($25). This cost could be further controlled through a rental service. The fabrication materials, *i.e.*, transparency and liquid silicone, can be purchased in bulk for under $20. As far as we know, our method is arguably the most cost-effective and precise fingerprint fabrication method at present.

Except for the silicone curing process (15 minutes to 6 hours depending on the temperature), all other operations involved to get MaskPrints can be completed within 10 minutes, *e.g.*, scanning and generating MaskPrints, printing them, pouring liquid silicone, and trimming the cured MaskPrints. It is worth noting that fabricating MaskPrint is a very low-frequency activity and can be outsourced to a trusted service provider. For a target FRS, the enrollment process takes approximately several minutes. In comparison, a typical fingerprint enrollment without using MaskPrint takes about 30 seconds. It should be noted that enrollment is also a low-frequency operation. Users use their fingers to interact with the FRS in their daily activities.

Table 1: Participant Demographics.

Participant	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
Gender	M	M	M	F	F	M	M	M	M	M	M	F	F	F	F	F	F	F	M	F
Age	19 - 22					23 - 28													>50	
Profession	Uundergraduate					Postgraduate													Staff	

Table 2: Number of Minutiae of Participating Fingers.

(a)

Finger	A_{R2}	B_{R2}	C_{R2}	D_{R2}	E_{R2}	F_{R2}	G_{R2}	H_{R2}	I_{R2}	J_{R2}
Minutiae#	69	81	82	44	56	66	54	57	64	56
Centering#	49	70	55	32	43	51	39	49	41	35
Marginal#	20	11	27	12	13	15	15	8	23	21

(b)

Finger	K_{R2}	L_{R2}	M_{R2}	N_{R2}	O_{R2}	P_{R2}	Q_{R2}	R_{R2}	S_{R2}	T_{R2}
Minutiae#	65	74	76	79	57	61	73	65	80	87
Centering#	48	54	66	65	44	48	61	40	52	50
Marginal#	17	20	10	14	13	13	12	25	28	37

(c)

Finger	F_{R0}	F_{R1}	F_{R3}	F_{R4}	F_{L0}	F_{L1}	F_{L2}	F_{L3}	F_{L4}	D_{R1}	S_{R1}
Minutiae#	77	65	67	69	88	68	60	69	57	38	68
Centering#	62	55	53	59	68	55	50	57	49	33	57
Marginal#	15	10	14	10	20	13	10	12	8	5	11

6.2 FRS Testbeds

Two FRSs are used to evaluate MaskPrint. They sense fingerprints through the same optical scanner (ZKTeco Live 20R) and process the fingerprint images with ZKFinger SDK 5.3 from ZKTeco and VeriFinger 12.4 from Neurotechnology, respectively. ZKFinger is widely used in attendance machines[3] and VeriFinger has been deployed in financial and government applications[4]. They are both widely adopted by work in the field [14, 32, 37].

VeriFinger represents the top-notch commercial fingerprint recognition algorithms, while ZKFinger is an example of the relatively mediocre solutions. We only implement the enrollment and identification processes with the Application Programming Interface (APIs) of two SDKs, and leave any settings as default, *e.g.*, the matching thresholds. The fingerprint recognition algorithms of the two SDKs are black boxes to us. We do not have any knowledge about how they process images and generate templates, nor do we know how they match two templates. The information we can obtain from both of them is the matching scores and the matching results. VeriFinger provides more information, such as the extracted minutiae data and which minutiae are matched between the two templates.

6.3 Participants

We recruited 20 participants of different backgrounds to validate the effectiveness of MaskPrint. Their detailed information is listed in Table 1. In the rest parts, we use letters **A-T** to indicate the 20 participants, and **R0-R4** and **L0-L4** to indicate their thumb, index,

[3] ZKTeco's product NGTeco is the best-selling fingerprint attendance machine on Amazon.

[4] The applications of VeriFinger are presented on its website, https://neurotechnology.com/verifinger-references.html.

middle, ring, and little fingers of the right and left hands. P_{Fi} is used to indicate participant P's finger Fi, *e.g.*, A_{R2} is participant A's right middle finger. Participant A-T's right middle fingers[5] are used to evaluate MaskPrint among different people, while participant F's ten fingers and participant D and S's right index fingers are used to evaluate with respect to one user's different fingers. The dataset involves 31 fingers in total. The distribution of their minutiae numbers is listed in Table 2.

All information collection and experimental protocols have been approved by the Ethical Review Board of the authors' institute. Additionally, the fingerprint data is encrypted and stored offline. The physical MaskPrints are securely locked and will be destroyed after the tests. The decryption password and the unlock key are only accessible to the authors. Participants are provided with full disclosure about our study.

6.4 Methodology

To thoroughly evaluate MaskPrint's usability and effectiveness in privacy protection, we conduct the following experiments. We first measure the number of minutiae required for enrollment in different FRSs, which reflects, on one hand, the time cost of enrollment, and on the other hand, the heterogeneity in different FRS algorithms (Section 7.1).

With the enrolled FRSs, we conduct three tests to assess the security of MaskPrint and its impact on usability. The finger from which the enrolled MaskPrint is derived is defined as the *Source Finger*. MaskPrints derived from the same finger are defined as *Sibling MaskPrints*. We also define the following performance metrics:

- Genuine Attempt: an attempt to test a finger against an FRS that was enrolled with the finger or the finger's MaskPrint.
- Imposter Attempt: an attempt to test a finger against an FRS that was not enrolled with the finger or the finger's MaskPrint; or an attempt to test a MaskPrint, *e.g.*, a forged fingerprint based on the data leaked from another FRS, against an FRS that enrolled with the MaskPrint's sibling MaskPrint.
- False Non-Match Rate (FNMR): the percentage of rejected genuine attempts.
- False Match Rate (FMR): the percentage of accepted imposter attempts.

Then, specifically, Section 7.2 measures the FNMR of source fingers to validate the usability; Section 7.3 measures the FMR of sibling Maskprints to assess the effectiveness of the protection mechanism; Section 7.4 measures the FMR of non-source fingers to further evaluate usability and security against brute-force attacks.

7 RESULTS

In this section, we present the evaluation results.

7.1 Enrollment

We generate 10 batches of MaskPrints for each finger by selecting different minutiae sets for different batches. As shown in Figure 12, each batch contains 7 groups of 3 MaskPrints with an increasing number (from 10 to 40 in step 5) of minutiae. We use the method in Section 5.3 to determine the minimum number of minutiae, and record the results of the 10 batches in Table 3 and 4. Before testing with each batch, we first empty the fingerprint database to avoid interference from previous batches. We have the following findings.

[5]Testing with the middle finger is relatively convenient, and its fingerprint information is comparatively less sensitive (thumbs and index fingers have been widely enrolled in FRSs).

Table 3: Success Batches @ Template with the Minimum Required Number of Minutiae, ZKFinger

(a)

Finger	A_{R2}	B_{R2}	C_{R2}	$\mathbf{D_{R2}}$	E_{R2}	F_{R2}	G_{R2}	H_{R2}	I_{R2}	J_{R2}
Minutiae#	30	40	40	**35**	30	25	25	20	30	25
Success#	10/10	8/10	7/10	**3/10**	8/10	9/10	9/10	10/10	7/10	10/10

(b)

Finger	K_{R2}	L_{R2}	M_{R2}	N_{R2}	O_{R2}	P_{R2}	Q_{R2}	R_{R2}	$\mathbf{S_{R2}}$	T_{R2}
Minutiae#	30	40	30	35	20	30	25	30	**40**	40
Success#	10/10	8/10	7/10	10/10	8/10	9/10	10/10	7/10	**3/10**	10/10

(c)

Finger	F_{R0}	F_{R1}	F_{R3}	$\mathbf{F_{R4}}$	F_{L0}	F_{L1}	F_{L2}	F_{L3}	F_{L4}	D_{R1}	S_{R1}
Minutiae#	30	25	30	**40**	35	30	20	30	35	15	30
Success#	8/10	10/10	10/10	**2/10**	8/10	9/10	8/10	9/10	9/10	10/10	7/10

Table 4: Success Batches @ Template with the Minimum Required Number of Minutiae, VeriFinger.

(a)

Finger	A_{R2}	B_{R2}	C_{R2}	D_{R2}	E_{R2}	F_{R2}	G_{R2}	H_{R2}	I_{R2}	J_{R2}
Minutiae#	10	10	10	10	10	10	10	10	10	10
Success#	9/10	10/10	9/10	10/10	9/10	10/10	10/10	9/10	9/10	9/10

(b)

Finger	K_{R2}	L_{R2}	M_{R2}	N_{R2}	O_{R2}	P_{R2}	Q_{R2}	R_{R2}	S_{R2}	T_{R2}
Minutiae#	10	10	10	10	10	10	10	10	15	10
Success#	10/10	10/10	9/10	10/10	10/10	8/10	10/10	9/10	9/10	7/10

(c)

Finger	F_{R0}	F_{R1}	F_{R3}	F_{R4}	F_{L0}	F_{L1}	F_{L2}	F_{L3}	F_{L4}	D_{R1}	S_{R1}
Minutiae#	10	10	10	10	10	10	10	10	10	10	10
Success#	9/10	10/10	10/10	9/10	9/10	10/10	10/10	9/10	10/10	10/10	10/10

1). **ZKFinger requires more minutiae than VeriFinger to achieve a similar success rate.** ZKFinger requires 25 to 35 minutiae for most fingers, meaning that the user needs 4 to 6 enrollment attempts. At the same time, VeriFinger only needs 10 minutiae[6], *i.e.*, only one attempt is required. The reason for this difference is rooted in their algorithms' capability. A more capable algorithm can tolerate more missing minutiae.

2). **Fingerprint quality also affects matching results.** We observe that the MaskPrints of fingers D_{R2}, S_{R2}, and F_{R4} (marked in bold in Table 3) have a higher number of minutiae, yet their corresponding success rates remain relatively low. Upon examining their input images, we identify another factor independent of minutiae: fingerprint quality. The ridges in the original fingerprint images of these MaskPrints are discontinuous and blurred. One possible reason could be daily abrasions in the working environment. As a very special case, S_{R2} had been cut off and then connected back, which disrupted the ridge structure in the central area. Both abrasions and serious injuries would result in an inherently vague and irregular pattern in ridges and valleys. We consider such fingerprints to be of

[6]Figure 4 shows that more than 10 minutiae are required for VeriFinger. This is because we do not filter out spurious minutiae as SuperDB does (Section 5.1.2) in that test.

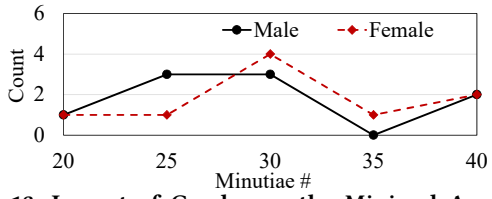


Figure 13: Impact of Gender on the Minimal Amount of Minutiae, ZKFinger.

low quality, making it difficult for ZKFinger to consistently extract canonical minutiae. In contrast, although D_{R1} has a similar number of minutiae and fingerprint area as D_{R2}, it succeeds with only 15 minutiae due to its high quality, *i.e.*, D_{R1}'s ridges and valleys are continuous with high contrast.

3). **Gender has no obvious impact.** We count the fingers by minutiae number and gender in Table 3(a) and Table 3(b), and plot the results in Figure 13. Both females and males need a similar amount of minutiae to achieve comparable success rates.

7.2 Testing with Source Fingers

The successfully enrolled templates in Section 7.1 are exported and kept for evaluation. In this test, for each template, we import it into an emptied FRS database and measure matching results with the corresponding source finger. For each source finger, 60 images are collected as the test input. They are fed into the FRSs by invoking their APIs. Depending on the number of successfully enrolled batches, each finger would undergo 420 to 600 genuine attempts. Table 5 shows that the FNMR values are all lower than 3%. As for a comparison, when using full finger templates, *i.e.*, without using MaskPrint, the FNMR values are 0% for both FRSs. The results indicate that, as MaskPrint reduces redundancy in the templates, using it has an impact on recognition accuracy (false rejection), but the low occurrence probability might not affect practical use.

Performance over Time. Fingerprints might slightly change over time. To evaluate the impact of this factor, we collect images of participant F's 10 fingers again after 5 months and repeat the above test. We find that VeriFinger's FNMR is very stable, while ZKFinger's FNMR slightly increases. As shown in Figure 14, the FNMR results are still lower than 7%, which means the system is still usable after 5 months. To maintain consistent performance, the user can generate new MaskPrints by masking the same minutiae set on newly collected fingerprint images and re-enroll into the target FRS every six months or a year.

Impact of Input Proficiency. The decreased FNMR results of F_{R1} and F_{L1} in Figure 14 reveal another time-related impacting factor: Proficiency. By analyzing the fingerprint images of Participant F over the 5 months, we find the reason is that this participant uses the two fingers for many experiments. Proficient input techniques can make the input image more stable and include more complete information.

7.3 Testing with Sibling MaskPrints

To evaluate the information protection capability of MaskPrint, we consider a typical scenario: an adversary has obtained the template information from a compromised FRS and generated the corresponding fingerprint image A to spoof a victim FRS, whose enrolled

Table 5: FNMR Results of the Source Finger

(a)

Finger	A_{R2}	B_{R2}	C_{R2}	D_{R2}	E_{R2}	F_{R2}	G_{R2}	H_{R2}	I_{R2}	J_{R2}
ZKFinger	0.21	1.96	2.35		1.84	0.46	2.46	1.81	1.05	2.12
VeriFinger	0.44	0.42	0.19	0.0	0.53	0.17	0.0	0.0	1.57	0.0

(b)

Finger	K_{R2}	L_{R2}	M_{R2}	N_{R2}	O_{R2}	P_{R2}	Q_{R2}	R_{R2}	S_{R2}	T_{R2}
ZKFinger	1.73	2.78	1.79	1.22	2.15	1.43	0.76	1.47		2.02
VeriFinger	0.0	0.0	1.14	0.5	0.0	0.11	0.17	1.57	0.26	0.95

(c)

Finger	F_{R0}	F_{R1}	F_{R3}	F_{R4}	F_{L0}	F_{L1}	F_{L2}	F_{L3}	F_{L4}	D_{R1}	S_{R1}
ZKFinger	1.24	0.69	0.6		0.89	2.19	0.25	2.56	1.74	1.88	0.4
VeriFinger	0.37	0.0	0.17	0.0	0.19	0.0	0.17	0.0	0.17	0.0	0.25

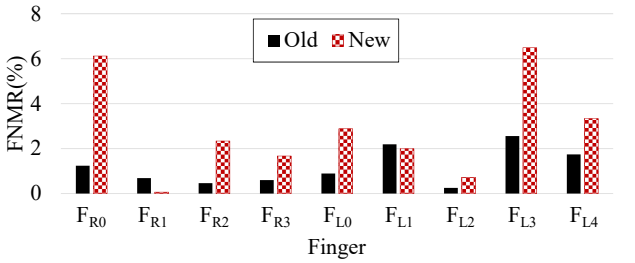


Figure 14: FNMR Results over Time, ZKFinger.

template is extracted from fingerprint image B. Since both FRSs are protected by MaskPrint, B and A are sibling MaskPrints.

In this test, we generate two sibling MaskPrints. One is used to spoof a victim FRS where another MaskPrint has been enrolled. Depending on the SDKs adopted by the compromised FRS and the victim FRS, four situations are considered, *i.e.*, fingerprint data leaked from a ZKFinger/VeriFinger FRS is used to spoof another ZKFinger/VeriFinger FRS. We denote the four situations with S_{ZZ}, S_{VZ}, S_{ZV}, and S_{VV}. For example, S_{ZV} represents the situation (S) where the adversary exploits data leaked from a ZKFinger FRS (Z) to spoof a VeriFinger FRS (V).

For each situation, five pairs of sibling MaskPrints are generated to repeat the above test five times. The minutiae sets of the five pairs are independently and randomly selected, but ensure that the minutiae sets of each pair of sibling MaskPrints are distinctive from each other. The number of minutiae in a MaskPrint is decided by the finger and target SDK according to Table 3 and 4. For example in S_{ZV}, if A_{R2} is the source finger, a MaskPrint of A_{R2} is generated with 30 minutiae and enrolled in the ZKFinger FRS; another MaskPrint with 10 different minutiae is generated and enrolled in the VeriFinger FRS.

Besides, to offer a baseline for this test, the full fingerprint images collected previously are used as a template to simulate the scenario without MaskPrint protection. As shown in Table 6(a), without the protection, the adversary can easily pass the identification of the victim FRS.

With the protection of MaskPrint, the FMR values in all situations are decreased (Table 6(b)). ZKFinger's FMR values are 0.79% and 0% in S_{ZZ} and S_{VZ}, while VeriFinger's FMR values are 50.12% and 28.41% in S_{ZV} and S_{VV}, respectively. This difference in FMR value between ZKFinger and VeriFinger is caused by their different

Table 6: FMR Results of (a) Full Fingerprints and (b) Sibling MaskPrints.

		Presented to	
	FMR(%)	ZKFinger	VeriFinger
Leaked from	ZKFinger	100	100
	VeriFinger	100	100

(a)

		Presented to	
	FMR(%)	ZKFinger	VeriFinger
Leaked from	ZKFinger	0.79	50.12
	VeriFinger	0	28.41

(b)

Table 7: FMR and FNMR with Different Thresholds

Threshold		48	60	72	84	96
FMR (%)	ZKFinger -> VeriFinger	50.12	35.52	18.4	8.33	3.53
	VeriFinger ->VeriFinger	28.41	10.32	3.86	1.08	0.65
VeriFinger FNMR (%)		0.31	6.3	17.35	30.58	44.42

matching capabilities. We found that VeriFinger is able to match two fingerprints even when there is no compatible minutiae pair in two templates in S_{VV}. It seems that the slightly overlapped non-minutiae area is almost enough for it to make correct decisions. In S_{ZV}, the leaked template contains more fingerprint information, thus, which further increases the FMR value. The results indicate that when FRSs of different capabilities are mixed in use, a weaker FRS may lead to more information leakage. Tailoring mask designs to specific FRS algorithms should be able to avoid this issue.

Impact of Matching Thresholds. In Table 6, the results of VeriFinger are obtained with the default threshold (48). Higher thresholds lead to a higher security level. We list the FMR and FNMR results with different thresholds in Table 7. By setting a higher threshold, FMR is reduced from 50.12% / 28.41% to 3.53% / 0.65% for S_{ZV} and S_{VV}. However, FNMR is increased from 0.31% to 44.42%. The above results reflect the trade-off between usability and security. With a given threshold, increasing the number of minutiae can reduce FNMR.

7.4 Testing with Non-source Fingers

Due to the reduction of minutiae in the FRS template after using MaskPrint, a potential side effect is the possibility of increased FRS recognition errors. To investigate this, we synthesized 200,000 fingerprints using Anguli [2]. The resolution and size of the synthetic fingerprint images are consistent with those collected by the scanner described in Section 6.1. The experimental protocols mirror those used in the source finger test in Section 7.2, with the replacement of the source finger images with synthetic ones.

The result is that both FRSs never accept any imposter attempts with/without applying MaskPrint. Since only VeriFinger returns the match scores of rejected attempts, we present the CDF (Cumulative Distribution Function) of these scores in Figure 15. It shows that, compared to full fingerprints, MaskPrint increases the overall matching scores. However, 99% of the scores are lower than 10 and the maximum score is 34. They are all lower than the default threshold. for the full fingerprints, 99% of the scores are lower than 4 and the maximum score is 26. The results indicate that the use of MaskPrint does not significantly increase the FMR and can withstand brute-force attacks.

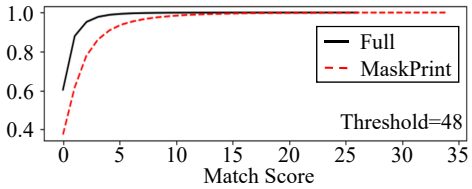


Figure 15: CDF of Match Scores against Non-source Fingers, VeriFinger.

8 DISCUSSION

In the following, we discuss the limitations of MaskPrint and provide directions for future research.

8.1 Estimation Before Enrollment

In the current design, the users should try to enroll first, and determine whether their fingers are suitable to generate MaskPrint. However, as shown in Section 7.2, ZKFinger cannot stably identify finger D_{R2} and F_{R4} after masking a small part of minutiae. This would cause annoying futility. Thus, estimation of fingerprint state would be helpful to offer advice about whether the finger is suitable for applying MaskPrint, and should choose another finger. Further, the user can provide more information about the FRS, *e.g.*, the vendor. Then, it is possible to reduce the number of enrollment attempts by estimating how many minutiae is a proper value, instead of starting from 10 minutiae.

8.2 Protection on Personal Devices

The current MaskPrint design is not suitable for some personal devices due to several reasons. First, some personal devices deploy capacitive fingerprint sensors that cannot sense non-conductive silicone-made MaskPrints. This issue can be addressed by applying a conductive coating or replacing silicone with other casting material (*e.g.*, polyvinyl acetate glue [32]) to fabricate physical MaskPrint. Another problem is that some devices adopt adaptive matching algorithms to provide better resilience to gradual changes in fingerprint information, which means they might learn and adapt to the full fingerprint. To deal with this problem, the user may need to explicitly disable this function or re-enroll MaskPrint more frequently.

9 RELATED WORK

Fingerprint information is considered very sensitive because it rarely changes; once leaked, it loses its privacy forever. Recent work demonstrates injection attacks [17] and presentation attacks [24, 32, 33] with fingerprint information.

Plenty of work has been devoted effort to protecting fingerprint information from leakage. Trusted Execution Environments (TEEs) are proposed to protect sensitive information from root attackers [3, 26, 28]. It isolates all I/O and processing modules associated with fingerprint data from the operating system. However, the vulnerabilities are reported that it is possible to extract sensitive data from TEEs [16, 34].

To prevent the exploitation of fingerprint information even after it has been leaked, cancelable fingerprint templates and fingerprint cryptosystems are proposed to store and match fingerprints after

noninvertible processing. Cancelable templates transform the fingerprint template into another representation space [27, 31, 35], such as transforming the minutiae coordinates and directions with a secret surface folding function [30]. Meanwhile, fingerprint cryptosystems reliably extract cryptographic keys from noisy fingerprint data by utilizing error correction coding schemes [18, 29]. Both the transformed fingerprint data and the extracted keys cannot be used to reconstruct the original data, thus, through setting different transforming/extracting parameters in the FRSs, they can avoid further exploitation of leakage.

However, all of these countermeasures require either software or hardware support of FRSs, which is not only nontransparent to the end-users, but also impractical to be deployed on every FRS. MaskPrint differs from existing work in its protection stage - protection at the pre-enrollment stage. It is also a user-transparent and device-agnostic fingerprint protection measure, which complements existing protection mechanisms.

10 CONCLUSION

The security of fingerprint data is more and more critical in modern life. A user would enroll one fingerprint into multiple FRSs. Once one of the FRSs is compromised and the recorded fingerprint data is leaked, it would be used to spoof the other FRSs, as they are enrolled with the same finger. This paper proposes MaskPrint, a user-transparent and device-agnostic protection measure against fingerprint database breaches. It achieves a good balance between usability and privacy protection.

ACKNOWLEDGMENTS

We thank anonymous CCS reviewers and Shepherd. Their valuable comments help us improve this paper. This work is supported by ShanghaiTech.

REFERENCES

[1] Over 27.8m records exposed in BioStar 2 data breach, 20220243. Available: https://www.trendmicro.com/vinfo/us/security/news/online-privacy/Over-27-8M-Records-Exposed-in-BioStar-2-Data-Breach.
[2] Anguli: Synthetic Fingerprint Generator, 2024. Available: https://dsl.cds.iisc.ac.in/projects/Anguli/index.html.
[3] Fingerprint HIDL, 2024. Available: https://source.android.com/docs/security/features/authentication/fingerprint-hal.
[4] FVC-onGoing, 2024. Available: https://biolab.csr.unibo.it/FvcOnGoing/.
[5] Hacker fakes German minister's fingerprints using photos of her hands, 2024. Available: https://www.theguardian.com/technology/2014/dec/30/hacker-fakes-german-ministers-fingerprints-using-photos-of-her-hands.
[6] Hackers can unlock a smartphone with fingerprints on glass of water, 2024. Available: https://www.hackread.com/hackers-unlock-smartphone-fingerprints-glass-of-water/.
[7] Indian police nab fraudsters who cloned fingerprints to spoof Aadhaar, 2024. Available: https://www.biometricupdate.com/202302/indian-police-nab-fraudsters-who-cloned-fingerprints-to-spoof-aadhaar.
[8] iPhone 5S fingerprint sensor hacked by Germany's Chaos Computer Club, 2024. Available: https://www.theguardian.com/technology/2013/sep/22/apple-iphone-fingerprint-scanner-hacked.
[9] NIST Biometric Image Software, 2024. Available: https://www.nist.gov/services-resources/software/nist-biometric-image-software-nbis.
[10] OPM says 5.6 million fingerprints stolen in cyberattack, five times as many as previously thought, 2024. Available: https://www.washingtonpost.com/news/the-switch/wp/2015/09/23/opm-now-says-more-than-five-million-fingerprints-compromised-in-breaches/.
[11] UP: Man learns 'cloning fingerprints' online, 'hacks' 500 bank accounts, 2024. Available: https://timesofindia.indiatimes.com/city/bareilly/man-26-learns-cloning-fingerprints-online-hacks-nearly-500-accounts-with-bank-mitrass-help/articleshow/81158623.cms.
[12] VeriFinger, 2024. Available: https://neurotechnology.com/verifinger.html.
[13] R. Bouzaglo and Y. Keller. Synthesis and Reconstruction of Fingerprints using Generative Adversarial Networks, Mar. 2023. arXiv:2201.06164 [cs].
[14] K. Cao and A. K. Jain. Learning Fingerprint Reconstruction: From Minutiae to Image. *IEEE Transactions on Information Forensics and Security*, 10(1):104–117, Jan. 2015. Conference Name: IEEE Transactions on Information Forensics and Security.
[15] K. Cao and A. K. Jain. Hacking mobile phones using 2d printed fingerprints. *Dept. Comput. Sci. Eng., Michigan State Univ., East Lansing, MI, USA, Tech. Rep. MSU-CSE-16-2*, 6, 2016.
[16] D. Cerdeira, N. Santos, P. Fonseca, and S. Pinto. Sok: Understanding the prevailing security vulnerabilities in trustzone-assisted tee systems. In *2020 IEEE Symposium on Security and Privacy (SP)*, pages 1416–1432, 2020.
[17] Y. Chen, Y. Yu, and L. Zhai. {InfinityGauntlet}: Expose smartphone fingerprint authentication to brute-force attack. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 2027–2041, 2023.
[18] Y. Dodis, L. Reyzin, and A. Smith. Fuzzy extractors: How to generate strong keys from biometrics and other noisy data. In *Advances in Cryptology-EUROCRYPT 2004: International Conference on the Theory and Applications of Cryptographic Techniques, Interlaken, Switzerland, May 2-6, 2004. Proceedings 23*, pages 523–540. Springer, 2004.
[19] J. J. Engelsma, K. Cao, and A. K. Jain. Learning a fixed-length fingerprint representation. *IEEE transactions on pattern analysis and machine intelligence*, 43(6):1981–1997, 2019.
[20] L. B. Felsen. Evanescent waves. *JOSA*, 66(8):751–760, 1976.
[21] J. Feng. Combining minutiae descriptors for fingerprint matching. *Pattern Recognition*, 41(1):342–352, 2008.
[22] C. Hill. Risk of Masquerade Arising from the Storage of Biometrics. 2001.
[23] Z. Hu, D. Li, T. Isshiki, and H. Kunieda. Hybrid minutiae descriptor for narrow fingerprint verification. *IEICE TRANSACTIONS on Information and Systems*, 100(3):546–555, 2017.
[24] C. Kauba, L. Debiasi, and A. Uhl. Enabling fingerprint presentation attacks: Fake fingerprint fabrication techniques and recognition performance. *arXiv preprint arXiv:2012.00606*, 2020.
[25] Y. C. Ko, L. Melani, N. Y. Park, and H. J. Kim. Surface characterization of paper and paperboard using a stylus contact method. *Nordic Pulp & Paper Research Journal*, 35(1):78–88, 2020.
[26] T. Mandt, M. Solnik, and D. Wang. Demystifying the Secure Enclave Processor.
[27] C. Moujahdi, G. Bebis, S. Ghouzali, and M. Rziza. Fingerprint shell: Secure representation of fingerprint template. *Pattern Recognition Letters*, 45:189–196, 2014.
[28] B. Ngabonziza, D. Martin, A. Bailey, H. Cho, and S. Martin. Trustzone explained: Architectural features and use cases. In *2016 IEEE 2nd International Conference on Collaboration and Internet Computing (CIC)*, pages 445–451, 2016.
[29] S. Rane, Y. Wang, S. C. Draper, and P. Ishwar. Secure biometrics: Concepts, authentication architectures, and challenges. *IEEE Signal Processing Magazine*, 30(5):51–64, 2013.
[30] N. K. Ratha, S. Chikkerur, J. H. Connell, and R. M. Bolle. Generating cancelable fingerprint templates. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 29(4):561–572, 2007.
[31] N. K. Ratha, J. H. Connell, and R. M. Bolle. Enhancing security and privacy in biometrics-based authentication systems. *IBM systems Journal*, 40(3):614–634, 2001.
[32] A. S. Rathore, Y. Shen, C. Xu, J. Snyderman, J. Han, F. Zhang, Z. Li, F. Lin, W. Xu, and K. Ren. FakeGuard: Exploring haptic response to mitigate the vulnerability in commercial fingerprint anti-spoofing. In *NDSS*, 2022.
[33] C. W. Schultz, J. X. Wong, and H.-Z. Yu. Fabrication of 3d fingerprint phantoms via unconventional polycarbonate molding. *Scientific reports*, 8(1):9613, 2018.
[34] A. Shakevsky, E. Ronen, and A. Wool. Trust dies in darkness: Shedding light on samsung's TrustZone keymaster design. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 251–268, Boston, MA, Aug. 2022. USENIX Association.
[35] A. Teoh, A. Goh, and D. Ngo. Random multispace quantization as an analytic mechanism for biohashing of biometric and random identity inputs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 28(12):1892–1901, 2006.
[36] T. Uz, G. Bebis, A. Erol, and S. Prabhakar. Minutiae-based template synthesis and matching for fingerprint authentication. *Computer Vision and Image Understanding*, 113(9):979–992, 2009.
[37] K. P. Wijewardena, S. A. Grosz, K. Cao, and A. K. Jain. Fingerprint template invertibility: Minutiae vs. deep templates. *IEEE Transactions on Information Forensics and Security*, 18:744–757, 2023.
[38] Z. Zhang. A flexible new technique for camera calibration. *IEEE Transactions on pattern analysis and machine intelligence*, 22(11):1330–1334, 2000.
[39] R. Zhou, D. Zhong, and J. Han. Fingerprint identification using sift-based minutia descriptors and improved all descriptor-pair matching. *Sensors*, 13(3):3142–3156, 2013.

APPENDIX

A EXAMPLES OF NON-IDEAL IMPRESSIONS

Figure 16(b)-16(d) present three non-ideal impressions of the same finger where Figure 16(a) is captured from. It is obvious that only a part of the information in the images can be used to match them with each other.

(a) Good Quality (b) Wet (c) Dry (d) Incomplete

Figure 16: Fingerprint images captured by optical sensor under different conditions.